SHIKSHANTAR ACADEMY SECONDARY SCHOOL

Bastipur, Karjanha-05, Siraha, Madhesh Province, Nepal | Estd: 2072 B.S.
Affiliated to National Examinations Board (NEB) & Ministry of Education, Nepal
Phone: +977 9807790805, +977 9709867048 | Email: shikshantaracademy@gmail.com

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Academic Standard: IEEE 830-1998 Compliant | Field: Computer Science & Engineering (CSE)

System Name: Shikshantar Academy School Management & Cash Accounting System (SAMS)

Specification Attribute	Project Implementation Detail
System Title	Shikshantar Academy School Management System (SAMS)
Architecture	Single-Page Application (SPA) with Cloud-Native Reactive Datastore
Front-End Technology	React 18, TypeScript, Tailwind CSS, Vite, jsPDF, Lucide Icons
Back-End & Database	Google Cloud Firestore (NoSQL Document Database), Firebase Auth
Target Academic Year	Session 2083 B.S. (Bikram Sambat) & 2026-2027 A.D.
Primary Domain	Physical Cash Desk Accounting, Student Ledgers, Batch Invoicing, RBAC
Prepared For	Senior School Management, CSE Academic Reviewers, Systems Auditors

1. INTRODUCTION & SYSTEM OVERVIEW

The Shikshantar Academy School Management System (SAMS) is an institutional ERP tailored specifically to the operational, economic, and regulatory needs of schooling in Nepal. Operating on a dual calendar system (Bikram Sambat B.S. and Gregorian A.D.), SAMS bridges traditional paper-based ledger registers (Khatas) with high-integrity digital ledger tracking.

- **Purpose**: Establish definitive functional, structural, and security constraints for academic management and physical cash desk operations.
- **Scope**: Encompasses Master Fee Tariffs, Monthly Batch Billing, Counter Cash Desk, Student Ledger Accounts, Daily Reversal Registers, and Immutable Forensic Audit Trails.
- **Target Users**: Super Administrator (Principal), Chief Accountant, Counter Cashier, Faculty Members, Guardians, and Students.

2. SYSTEM ARCHITECTURE & CSE DESIGN PATTERNS

From a Computer Science & Engineering standpoint, SAMS is structured around modern architectural patterns to ensure high responsiveness, consistency, and zero phantom calculations:

- **Flux / CQRS Pattern**: Separates transactional command dispatches (WriteBatch) from reactive query listeners (Firestore onSnapshot). State updates reflect immediately without full page reloads.
- **Distributed Atomic Transactions**: Multi-document mutations (such as recording a receipt, marking an invoice paid, and updating student debt) are wrapped inside atomic WriteBatches guaranteeing ACID compliance.

- **Compensatory Event Sourcing (Undo Engine):** Transactions are never destructively dropped or hard-deleted. Corrections are achieved by writing compensatory rollback states and flagging records with isVoid=true, preserving total auditability.

3. DATABASE ENTITIES & COLLECTIONS (DATA DICTIONARY)

The backend uses Google Cloud Firestore organized into normalized collections optimized for relational integrity within a document datastore:

Collection / Table	Key Entity Type	Primary Key Structure	Cardinality & Role
users	Student, Teacher, Staff	UID / Student ID	1:N with transactions and fee records
feeStructure	Master Tariffs	academic_fees (Singleton)	Defines class-wise tuition, bus routes, lab fees
studentFees	Monthly Invoices	{studentId}_{nepaliMonth}	Holds monthly billed dues, arrears, and paid status
feeTransactions	Cash Receipts	REC-{Year}-{Serial}	Represents physical cash intake at the desk
feeAuditLogs	Forensic Audit Trail	UUID (Immutable)	Captures who mutated what, timestamp, and diff snapshot
expenses	Outflow Vouchers	EXP-{UUID}	Tracks cash disbursements from counter drawer

3.1 Collection Schema: users (Students & Staff)

Field Name	Type	Null / Req	Description & Business Constraint
id	string	Required	Primary Key: Firebase Auth UID or Custom Roll/Registration ID
name	string	Required	Full legal name of the student or faculty member
role	enum	Required	Access tier: "admin" | "accountant" | "teacher" | "student"
class / studentClass	string	Optional	Academic grade (e.g., "PG", "Nursery", "Class 1" to "Class 10")
rollNo	string	Optional	Assigned class roll number
monthlyFee	number	Required	Base monthly tuition assigned to the student (synced from tariff)
busStop	string	Optional	Assigned route station (e.g., "Mirchaiya", "Bastipur")
transportFee	number	Optional	Monthly bus fare mapped from transportation structure
scholarshipStatus	string	Optional	"None" | "Provided" - indicates fee concession status
scholarshipAmount	number	Optional	Concession value in NRs or percentage concession
guardianPhone	string	Optional	10-digit mobile contact number for SMS receipts

3.2 Collection Schema: feeTransactions (Cash Receipts)

Field Name	Type	Null / Req	Description & Financial Purpose
id	string	Required	Receipt Serial (e.g., "REC-2083-0491") with auto-increment guarantee
studentId	string	Required	Foreign Key referencing users.id
studentName	string	Required	Snapshot of student name at time of payment
class	string	Required	Student grade level at transaction time
amountPaid	number	Required	Net cash received (> 0) entered into school drawer
cashTendered	number	Required	Physical currency note value handed by parent (>= amountPaid)
changeReturned	number	Required	Computed: cashTendered - amountPaid
khataPageNo	string	Optional	Cross-reference page number in physical paper registry book

Field Name	Type	Null / Req	Description & Financial Purpose
manualSlipNo	string	Optional	Serial number of manual carbon paper slip book
cashierId	string	Required	Authenticated UID of counter operator
nepaliDate	string	Required	Bikram Sambat transaction date (e.g., "2083-02-15")
isVoid	boolean	Required	Default false; flagged true if reversed via Undo Log
reversalReason	string	Nullable	Mandatory explanation string if receipt is voided

3.3 Collection Schema: studentFees (Invoices) & feeAuditLogs

Collection	Field	Type	Constraint / Validation Rule
studentFees	id	string	Compound Key: {studentId}_{nepaliMonth}
studentFees	totalFee	number	Tuition + Bus + Exam + Lab - Scholarship + Past Arrears
studentFees	paidAmount	number	Accumulated payments towards this bill; cannot exceed totalFee
studentFees	status	enum	"due" (0 paid), "partial" (partially settled), "paid" (fully settled)
feeAuditLogs	id	string	Primary Key: UUIDv4
feeAuditLogs	eventType	enum	"RECEIPT_ISSUED" | "RECEIPT_VOIDED" | "BASE_FEE_SYNCED" | "DISCOUNT"
feeAuditLogs	actorUid	string	UID of authenticated operator performing mutation
feeAuditLogs	previousState	map	Deep JSON copy of document state prior to mutation
feeAuditLogs	newState	map	Deep JSON copy of document state after mutation

4. DETAILED FUNCTIONAL REQUIREMENTS (FR)

Functional requirements define the core operational tasks executed by the system:

Req ID	Module / Component	Functional Requirement Statement & Acceptance Standard
FR-1.1	**Master Fee Tariffs**	System shall enable Super Admins to configure class-wise tuition, admission, exam, and computer fees across all levels (Playgroup to Class 10).
FR-1.2	**Route Management**	System shall maintain zone-wise bus tariffs mapped to pickup stops.
FR-1.3	**Base Fee Sync**	Clicking "Sync Base Fees" shall batch-update all student profile records with updated class tuition rates using atomic Firestore commits.
FR-1.4	**Official Tariff PDF**	System shall generate print-ready official PDF documents featuring the school crest logo, board affiliations, and authorized signature lines.
FR-2.1	**Remit Invoicing**	System shall execute batch billing for selected Nepali months, computing dues using tuition, transport, exam fees, and rolling forward unpaid arrears.
FR-2.2	**Concession Engine**	Invoicing engine must subtract approved scholarship percentages or fixed amounts before computing net monthly due.
FR-3.1	**Cash Desk Search**	Cashier can locate student by typing Roll, Name, or Grade in < 300ms.
FR-3.2	**Change Calculation**	System calculates exact change return: Change = Tendered - Paid, blocking negative change.
FR-3.3	**Khata Linking**	Mandates entry of physical ledger book page number and manual receipt serial.
FR-3.4	**Dual-Copy Receipts**	Renders dual copies (Student + Accounts) with amount in words in Nepalese Rupees.
FR-4.1	**Student Ledger**	Dynamically aggregates debits (billed) and credits (paid) with running balances.
FR-5.1	**Undo Engine**	Allows authorized reversal of erroneous entries, marking records as isVoid=true, restoring student debt, and recording an immutable audit log.

Req ID	Module / Component	Functional Requirement Statement & Acceptance Standard
FR-6.1	**Top Executive Bar**	Displays real-time Inflow (Gross Receipts), Outflow (Expenses), and Net Cash in Counter.

5. NON-FUNCTIONAL REQUIREMENTS (NFR)

NFR Category	Standard Metric	Detailed Implementation Requirement
Data Integrity	**ACID Compliance**	All multi-table balance mutations MUST execute via Firestore WriteBatches; no partial state mutations permitted.
Auditability	**Non-Destructive DB**	Zero hard deletes permitted on financial collections. Voided transactions retain complete forensic history.
Performance	**< 300 ms Lookup**	Student search and balance retrieval complete in under 300 ms over standard 4G/broadband connection.
Throughput	**500 students / 5s**	Batch invoice generation processes at least 100 students per second via chunked batch writes.
Client Speed	**< 1.5s PDF Render**	Official fee charts and dual-copy receipts generate entirely client-side via jsPDF with zero server CPU overhead.
Security	**RBAC & Firestore Rules**	Granular Firestore security rules restrict tariff edits to Admins and deny write operations on audit logs.
Localization	**Dual Calendar (B.S. & A.D.)**	All vouchers, daybooks, and reports display both Bikram Sambat and Gregorian dates alongside Nepalese currency formatting.
Fault Tolerance	**Offline Persistence**	Firestore offline persistence enabled to prevent data loss during momentary network disconnects.

6. VERIFICATION & VALIDATION TRACEABILITY MATRIX

Req ID	Target Module	Testing Methodology	Verification Criteria & Expected Outcome
FR-1.3	**Fee Tariff**	Integration Test	Altering Class 1 fee and executing Sync Base Fees updates all matching student user records.
FR-2.1	**Remit Invoicing**	Unit Calculation Test	Generating Jestha invoice accurately calculates Tuition + Bus - Concession + Past Arrears.
FR-3.2	**Cash Counter**	Boundary Analysis	Entering cash tendered < amount paid disables submit button and alerts cashier.
FR-3.4	**Receipt Engine**	Visual & Print Audit	Receipt correctly outputs words (e.g., "NRs. Three Thousand Five Hundred Only") with dual copies.
FR-5.1	**Undo Engine**	Negative / Regression Test	Reversing a transaction restores student debt, flags isVoid=true, and updates Top Executive Bar.
NFR-1.1	**Core Database**	Network Interruption Test	Simulating dropped connection mid-batch results in clean rollback without orphaned rows.

DOCUMENT APPROVAL & SIGN-OFF

Prepared By (CSE Analyst)

Reviewed By (Chief Accountant)

Approved By (Principal / Admin)